

HarnessSecurity-Bench: An Empirical Study and Benchmark of Security Mechanisms in Coding Agent Harnesses

ZHENGYANG ZHU, School of Software Engineering, Sun Yat-sen University, China and Peng Cheng Laboratory, China

LIMING HUANG, School of Software Engineering, Sun Yat-sen University, China

RUNMIN JI, East China Normal University, China

MINGXI YE, School of Software Engineering, Sun Yat-sen University, China

ZIHAN ZHOU, School of Software Engineering, Sun Yat-sen University, China

HANYANG GUO, School of Software Engineering, Sun Yat-sen University, China

JINGWEN WU, Department of Computer Science, Hong Kong Baptist University, China

YUHAN YE, Tsinghua University, China

YUMING FENG, Department of New Networks, Peng Cheng Laboratory, China

HONG-NING DAI, Department of Computer Science, Hong Kong Baptist University, China

ZIBIN ZHENG*, School of Software Engineering, Sun Yat-sen University, China

Coding agent harnesses mediate tool use and authorize actions, yet their security mechanisms and runtime effects remain incompletely characterized. We present *HarnessSecurity*, the first systematic empirical study and benchmark of open- and closed-source coding agent harnesses. First, we derive a ten-mechanism taxonomy and then assess 400 harness-mechanism cells using independent ratings by researchers and large language model (LLM) judges. We find that about half of confirmed mechanism implementations are opt-in, while closed-source harnesses exhibit substantial evidence gaps. Second, we introduce HARNESSSECURITY-BENCH, a benchmark of 23 tasks across five attack surfaces without sacrificing legitimate task requirements. Using separate deterministic oracles to measure task utility and attack effects with security setting comparisons, we evaluate nine mechanisms across six leading harnesses: Claude Code, Codex CLI, Gemini CLI, gptme, Qwen Code, and GitHub Copilot. Under a controlled LLM baseline GLM-5.2, we conduct 2,500 trials, recording 81,155 tool calls and over 2.2 billion tokens. Enabling auto-approve increases utility and raises attack success from 29.2% to 95.6%. Network isolation and read-only mode reduce attack effects with substantial utility losses, while command allowlisting and command denylisting reduce attack effects with a small utility loss and a utility gain, respectively. Task-level cases show that restrictions on a shared capability can obstruct both legitimate and malicious operations, and that allowed tools or commands can leave unauthorized operations reachable through alternative execution paths. Harness providers should make security settings verifiable, test alternative execution paths to protected operations, and assess attack effects alongside task utility and execution costs.

Additional Key Words and Phrases: Coding agents, agent harnesses, LLM agents, security benchmark, empirical study, software security

1 Introduction

Coding agents based on large language models (LLMs) have moved from demonstrations to production, with products such as Claude Code [4] and Codex CLI [37] supporting autonomous repository-level software development. Each agent couples an LLM with a *coding agent harness* that assembles context, mediates tool use, and authorizes actions [14, 33]. Harness design affects task success and resource costs even when the underlying model is held fixed [18, 52].

Expanding coding agents’ capabilities also introduces risks in how their harnesses handle untrusted inputs, authorize tool execution, and process repository data. Recent disclosures illustrate

*Corresponding author.

Table 1. Attack surface coverage and evaluation design in coding agent studies.

Evaluation scope		Attack surfaces						Evaluation design			
Study	# H.	Repo.	Instr.	Tool	Know.	Ext.	User	Utility	ASR	ON/OFF	Oracle
AIShellJack [31]	2	✗	✓	✗	✗	✗	✗	✗	✓	✗	Tr.
QueryIPI [54]	5	✗	✗	✓	✗	✗	✗	✗	✓	✗	Jdg.
IssueTrojanBench [48]	3	✓	✗	✗	✗	✓	✗	✗	✓	✗	St.+Tr.
Skill-Inject [47]	3	✗	✓	✗	✗	✗	✗	✓	✓	✗	Jdg.
PoisonedSkills [42]	4	✗	✓	✗	✗	✗	✗	✗	✓	✗	Tr.+Jdg.
SkillHarm [36]	4	✗	✓	✗	✗	✗	✗	✓	✓	✗	St.
RepoGuardBench [26]	2	✓	✓	✗	✗	✗	✗	✓	✓	✗	St.+Tr.
CodeIPI [49]	1	✓	✗	✗	✗	✗	✗	✓	✓	✗	St.+Tr.
RedCode-Exec [22]	3	✗	✗	✗	✗	✗	✓	✗	✓	✗	St.+Tr.
CapScope [12]	1	✓	✓	✓	✗	✗	✗	✓	✓	✗	St.
HSB (ours)	6	✓	✓	✓	✓	✓	✗	✓	✓	✓	St.+Evt.

Notation: ✓: evaluated; ✗: outside scope; # H.: evaluated harnesses; Repo.: task repository/workflow artifacts; Instr.: instructions/skill packages; Tool: tool interfaces; Know.: knowledge/memory; Ext.: external services/execution dependencies; User: direct input; Utility: task completion; ASR: attack success rate; ON/OFF: enabled and disabled comparison of harnesses’ native security mechanism setting. Oracle denotes the primary attack-success check (St.: state; Tr.: trace; Evt.: structured events; Jdg.: semantic judge.)

how these risks arise in practice. In particular, prompt injection through issue titles induced shell command execution in a mirror of Cline’s triage workflow [24], demonstrating how untrusted content can redirect agent actions. Moreover, OpenAI and Anthropic reported network isolation failures during cybersecurity evaluations [8, 38], while IDEsaster showed how agent file edits triggered unsafe behaviors in the surrounding IDE [32]. An investigation also reported that ZCode uploaded workspace snapshots containing Git history [19], raising confidentiality concerns about the scope of repository data transmitted by agent tools.

Prior work has studied agent security from several perspectives. First, surveys and empirical studies classify agent defenses, harness architectures, and execution controls, with an inspection of harness source code and public product materials [2, 11, 16, 25, 27, 44]. Second, agent security studies audit permissions and violations of information flows, evaluate attacks, defenses, and authorization policies in agent workflows [10, 12, 17, 29, 57]. Third, coding agent benchmarks also examine how adversarial instructions, tool descriptions, issues, and skills redirect development actions [31, 36, 47, 48, 54].

However, existing studies remain limited in their empirical assessment of native security mechanisms across coding agent harnesses. Architectural mapping and literature reviews do not systematically assess mechanism implementation and default settings across coding products [27, 44]. Empirical comparisons cover only three coding agents [25] or eleven source-accessible harnesses [11], leaving closed-source products underrepresented. Practitioner comparisons describe native settings but do not report independent implementation status [2, 16]. These limitations leave a gap in the *systematic empirical study* of mechanism implementation and default settings across open- and closed-source coding agent harnesses. Existing benchmarks also remain limited in their comparative evaluation of security mechanisms. IssueTrojanBench covers three coding agents on two Python repositories, while CapScope compares authorization policies within one Pi-based harness on five Python repair tasks [12, 48]. Other evaluations focus on attacks through particular resources, including instruction files, tool descriptions, and skills [31, 36, 47, 54]. Although some measure both task utility and attack outcomes [36, 47], they do not systematically isolate the effects of mechanism settings across harnesses and attack surfaces. These limitations leave a gap in the *comparative assessment of security mechanisms across coding harnesses and attack surfaces*. To bridge this research gap, we raise the following research questions (RQs):

RQ1. *What distinct security mechanisms can be identified in real coding agent harnesses?*

RQ2. To what degree are these mechanisms *implemented*?

RQ3. How do security mechanisms in coding agent harnesses affect *security and utility* during *adversarially augmented* task execution?

To address these research questions, we present *HarnessSecurity*, an empirical study of 40 coding agent harnesses: 27 open-source and 13 closed-source products. For RQ1, we derive a taxonomy of ten security mechanisms for assessment across coding agent harnesses. For RQ2, we assess all 400 harness-mechanism cells, recording evidence on implementation status, default settings, and supported configurations. These analyses in RQ1 and RQ2 establish the mechanism definitions and configurations used in the runtime evaluation. For RQ3, we introduce **HARNESSECURITY-BENCH** (HSB), a benchmark comprising 23 coding tasks spanning diverse programming languages, including Python, JavaScript, C#, C++, Go, and Ruby. HSB covers nine of the ten mechanisms across six harnesses. We augment task environments with misleading context and malicious resources across five attack surfaces (shown in A–E of Fig. 1). This augmentation preserves legitimate user instructions, task requirements, utility oracles, and valid paths to task completion. HSB provides separate deterministic oracles to measure task utility and attack effects, runs trials in isolated Docker containers, and compares enabled and disabled settings of native security mechanisms under the same execution environment. Table 1 compares HSB with prior coding agent evaluations across harness coverage, attack surfaces, comparison of security mechanism settings, and evaluation oracles.

The RQ1 study established a taxonomy of ten security mechanisms, and the RQ2 assessment confirmed implementation in 205 of 400 harness-mechanism cells and absence in 83, while 112 remain unresolved. The RQ3 evaluation comprised 2,500 trials under GLM-5.2 across six harnesses, recording 81,155 tool calls, over 2.2 billion native tokens, and 589.32 hours of cumulative agent execution time. The security and utility effects differ across mechanisms, harnesses, and tasks. Enabling auto-approve increased both utility and attack success, with attack success rising from 29.2% to 95.6%. Network isolation and read-only mode reduced attack success from 57.1% to 0.8% and from 8.3% to 0.8%, respectively, with utility losses of 24.5 and 34.0 percentage points. Command allowlisting and command denylisting reduced attack effects with a small utility loss and a utility gain, respectively. Task-level cases show that restricting a shared capability can block both legitimate and malicious operations, while alternative execution paths can leave unauthorized operations reachable. Harness development should test alternative execution paths to protected operations and evaluate security restrictions in terms of attack effects, legitimate task utility, and execution costs. We summarize our contributions as follows:

- **Security mechanism dataset and empirical study (RQ1–RQ2).** We contribute a dataset covering 40 coding agent harnesses (27 open-source and 13 closed-source) and a taxonomy of ten security mechanisms derived by analyzing the source code and official product materials collected in the dataset. We further assess implementation status, default settings, and configuration across all 400 harness-mechanism cells, showing that about half of confirmed implementations are opt-in and that evidence gaps are more prevalent among closed-source harnesses.
- **HARNESSECURITY-BENCH (RQ3).** We contribute a containerized benchmark for evaluating nine security mechanisms through 23 coding tasks. Task environments are augmented with misleading context and malicious resources across five attack surfaces while preserving legitimate task requirements and utility oracles. The benchmark compares enabled and disabled settings of native security mechanisms under the same execution environment, using separate deterministic oracles to assess task utility and attack effects.

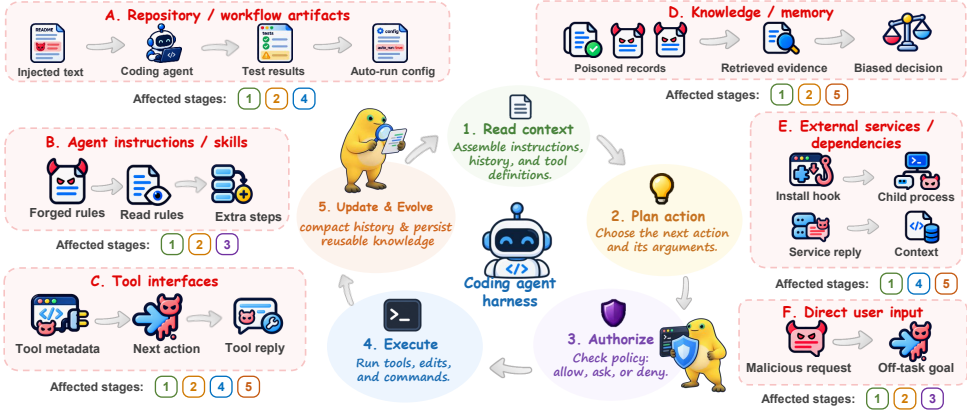


Fig. 1. Attack surfaces across the coding agent harness loop. The center shows five stages; panels A–F illustrate paths from adversarial resources or input to changed decisions or execution. Numbered markers identify stages that each surface can affect. HSB evaluates attacks on A–E.

2 Background

Fig. 1 places six attack surfaces around the five-stage *coding agent harness* loop. The harness first assembles instructions, repository context, interaction history, and tool definitions for the agent to read. The model plans an action, such as reading a file, editing code, or running a command. The harness then authorizes and executes or routes the request. The resulting interaction updates the context available in later turns [14, 18, 33]. Some harnesses also load reusable skills, delegate subtasks to subagents, and connect to external services through the Model Context Protocol (MCP), an open protocol for connecting LLM applications to external tools and data sources [5–7, 35].

At different stages of this loop, adversarial resources can influence agent decisions or harness execution. We use the *attack surface* to refer to a resource, interface, or state that an adversary can manipulate to produce such effects. Building on prior agent security research and frameworks [20, 25, 39, 57], we organize coding agent attack surfaces by the resource under adversarial control: *repository and workflow artifacts*, *agent instructions and skills*, *tool interfaces*, *knowledge and memory*, *external services and execution dependencies*, and *direct user input*, as shown in Fig. 1. Each surrounding panel shows how adversarial resources or input can affect the loop. Its numbered markers identify the stages involved.

3 Security Mechanisms Taxonomy (RQ1)

3.1 Harness Dataset

To construct the coding agent harness dataset for RQ1, we collected candidate harnesses from three channels: (i) GitHub repositories and lists of open-source coding agents; (ii) commercial products identified through vendor material; and (iii) coding agents that appeared in security disclosures and CVEs. The inventory contains 75 records (46, 17, and 12 from these three channels, respectively). After merging 19 pairs of records referring to the same product, we retained 56 unique candidates. For example, we merged two records for Amazon Q Developer CLI, one identified through repository mining and the other through a public security issue [3, 40].

To focus on widely adopted, actively maintained coding agent harnesses, we further screened these candidates. For open-source candidates, we applied four screening criteria: (1) the candidate had to be publicly released; (2) its GitHub repository had to have at least 1,000 stars; (3) it had to support an agentic coding workflow with code execution; and (4) publicly available materials had to allow us to assess its security mechanisms. We excluded projects with no maintenance for more than 12 months, as well as general agents and completion-only tools. For closed-source candidates, we required official documentation or CLI materials sufficient to assess their security mechanisms. Excluding 16 candidates, we finally retained 40 harnesses: 27 open-source and 13 closed-source (Table 2). The table distinguishes CLI tools, IDE extensions, IDE-based agents, and cloud/platform agents.

For the 40 retained harnesses, we designed a programmatic collection procedure to archive official web documentation, README files, and available CLI materials. The procedure followed official documentation indexes and sitemaps where available and selected operationally relevant pages using links, titles, and keywords. For open-source harnesses, we additionally collected source code, typically from official GitHub repositories. Notably, for Claude Code, we pinned an unofficial public mirror of source recovered from an npm package source map for RQ1 analysis [9]. GitHub Copilot was collected from the official VS Code repository [34]. The RQ1 dataset cutoff is 2026-08-31 (23:59:59 UTC). Per-harness snapshot metadata identify pinned commits while document manifests record capture dates, URLs, local paths, and SHA-256 hashes.

Table 2. The 40 harnesses studied.

	Harness	Stars (k)	Debut	Commits (k)	
Open-source	Aider	40	2023-05	9.0	
	Open Interpreter	58	2023-07	3.6	
	gptme	4	2023-08	2.5	
	Plandex	13	2024-04	2.2	
	SWE-agent	17	2024-04	3.2	
	Codebuff	1	2024-08	1.5	
	Goose	19	2024-11	5.5	
	Claude Code	—	2025-02	—	
	Codex CLI	28	2025-04	4.0	
	OpenCode	15	2025-04	5.0	
	Gemini CLI	60	2025-06	6.5	
	Qwen Code	27.4	2025-07	9.2	
	Pi Agent Harness	99.8	2025-08	6.2	
	Kimi Code CLI	7.2	2026-05	1.4	
	DeepSeek Harness	204.6	2026-08	14.6	
	Extension	Continue	25	2023-06	11.0
		Cline	42	2024-07	4.5
		Roo Code	16	2024-10	6.0
		Kilo Code	6	2025-02	3.5
	IDE	GitHub Copilot	—	2025-02	—
		PearAI	8	2024-09	15.0
		Void	25	2024-12	40.0
		Zed Agent	52	2025-01	55.0
	Platform	Refact.ai	2	2023-08	3.0
		Tabby	31	2023-09	4.8
		OpenHands	62	2024-03	8.5
		Bolt.new	13	2024-10	1.2
Closed-source	CLI	Warp Agent Mode	2024-06		
		Factory Droid	2025-03		
		Augment Code	2025-06		
	Extension	Tencent CodeBuddy	2025-04		
		JetBrains Junie	2025-04		
		Amp	2025-04		
	IDE	Cursor	2023-03		
		Windsurf	2024-11		
		Trae	2025-01		
		Kiro	2025-07		
		Google Antigravity	2025-11		
	Platform	Devin	2024-03		
		Google Jules	2025-05		

3.2 Study Design

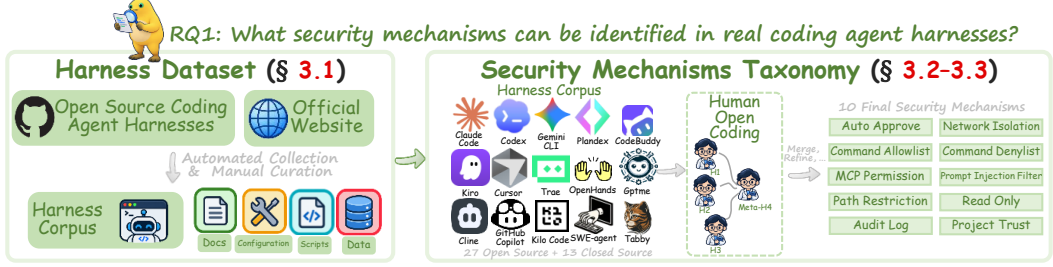


Fig. 2. Harness dataset construction and security mechanisms taxonomy in RQ1.

Following the principles of *open and axial coding* [50], we assigned initial security-related labels to eight harnesses, grouped the labels into mechanism categories, and then examined the remaining 32 harnesses to refine those categories and establish the final security mechanisms.

We selected eight initial harnesses covering open- and closed-source products and all four harness types: Aider, Claude Code, and Codex CLI (CLI agents), Cline and GitHub Copilot (IDE extensions), OpenHands and Devin (cloud/platform agents), and Cursor (IDE-based agent). Three researchers, each with at least seven years of software development experience, independently examined the official documentation, CLI references, and source code for eight harnesses in our collected dataset to identify security-related behavior and assign initial security-related labels. For each label, they recorded the associated security behavior and supporting evidence, including source-code locations where available, entry points, and configuration options. The same three researchers then examined the remaining 32 harnesses to extend the initial labels across the full dataset. For each observed behavior, they recorded supporting evidence and compared its protected object, enforcement point, and effect on the relevant operation with existing labels. They added labels for absence and consolidated overlapping labels. The three researchers then discussed the full labels with a fourth researcher who had at least ten years of experience in software development and research. The fourth researcher reviewed the evidence for each label and refined the mechanism categories based on what was protected and where checks occurred, and then adjudicated unresolved disagreements and made the final decisions on category boundaries. For example, labels concerning permitted file locations contributed to both path restriction and workspace boundary candidates. We included workspace boundaries under path restriction because workspace roots specify permitted file targets. Approval prompts were assessed through the auto-approve configuration contrast, while sandbox implementations were classified by the path, network, tool-permission, and approval checks they enforced.

3.3 Results

After initializing and extending the labels, three researchers retained 44 security-related labels (i.e., 16, 15, and 13). Following discussion and adjudication, four researchers grouped the labels into ten mechanisms: auto-approve (AA), network isolation (NI), command allowlisting (CAL), command denylisting (CDL), MCP permissions (MCP), prompt-injection filtering (PIF), path restriction (PR), read-only mode (RO), audit logging (AL), and project trust (PT). Table 3 defines the ten mechanisms and gives representative security settings with examples describing their common behaviors.

The taxonomy distinguishes mechanisms by the resources or operations they govern and their enforcement points in the harness loop (as shown in Fig. 1). PT determines whether repository-supplied configuration, instructions, and executable resources are admitted according to the project trust state. PIF tags, detects, or sanitizes untrusted instructions in file, web, or tool content. Among

Table 3. Ten security mechanisms and typical ON/OFF configurations.

Mech.	RQ1 definition	Typical configuration (ON/OFF)
AA	A mode that skips some or all confirmations (auto, YOLO, or full-auto); a risk capability that removes confirmation barriers.	Codex CLI : on: approval_policy="never" (no prompts or escalation); off: approval_policy="on-request" (approval requests allowed).
NI	Use default-deny egress or a domain allowlist to prevent unauthorized network access or exfiltration.	Codex CLI (workspace-write): sandbox_workspace_write.network_access=false (on) / true (off).
CAL	Match commands against an explicit allowed set to permit execution or bypass confirmation; unmatched commands may require approval or be denied.	Generic: on: default-deny command policy with an explicit allowlist; off: unrestricted command policy.
CDL	Block explicitly dangerous commands or patterns (for example, rm -rf or pipe-to-shell).	Claude Code: on: -disallowedTools="Bash (*)"; off: omit -disallowedTools.
MCP	Authorize MCP or external-tool servers and capabilities, and scope tool invocation.	Codex CLI: enabled_tools=[approved] and default_tools_approval_mode="prompt" (on); no tool allowlist and default_tools_approval_mode="approve" (off).
PIF	Tag, detect, or sanitize untrusted instructions carried by tool, web, or file content.	gptme (tool outputs) [21]: GPTME_INJECTION_HYGIENE=block (on) / off (off).
PR	Constrain file reads or writes to permitted paths; normalization and symlink handling depend on the implementation.	Codex CLI (write restriction; reads remain broadly allowed): on: sandbox_mode="workspace-write"; off: sandbox_mode="danger-full-access".
RO	Suppress persistent changes in the operations covered by a read-only, plan, or preview mode.	Codex CLI, with approval_policy="never" fixed: sandbox_mode="read-only" (on) / "workspace-write" (off).
AL	Record security-relevant actions and decisions for post-hoc forensics.	Qwen Code (session recording) [43]: general.chatRecording=true (on) / false (off).
PT	Admit repository-supplied configuration, instructions, and executable resources according to project trust state.	Codex CLI, project entry: trust_level="untrusted" (on) / "trusted" (off), with approval and sandbox settings fixed.

execution restrictions, PR specifies permitted file targets, whereas RO suppresses persistent changes in covered operations. A workspace write restriction can therefore allow edits within the workspace while preventing writes elsewhere. These distinctions identify which resources or operations a mechanism governs even when several controls share a single configuration mode.

The mechanisms also serve different security purposes. CAL, CDL, MCP, PR, NI, and RO govern the authorization or scope of operations. AA changes the confirmation requirements for those operations, and AL records actions and decisions for later review. This variation means that a mechanism’s presence must be interpreted together with its configuration and the operations it covers. RQ2 uses these categories to assess implementation and default settings, while RQ3 evaluates task utility and attack effects during execution.

Answer to RQ1. RQ1 identifies ten mechanisms spanning resource admission, content handling, operation authorization, execution restrictions, and audit logging. Their category boundaries reflect the resources and operations they govern and where they act in the harness workflow. AA changes confirmation requirements, while AL supports later review. The taxonomy provides common categories for assessing implementation, default settings, and runtime outcomes across harnesses.

4 Security Mechanism Empirical Study (RQ2)

4.1 Study Design

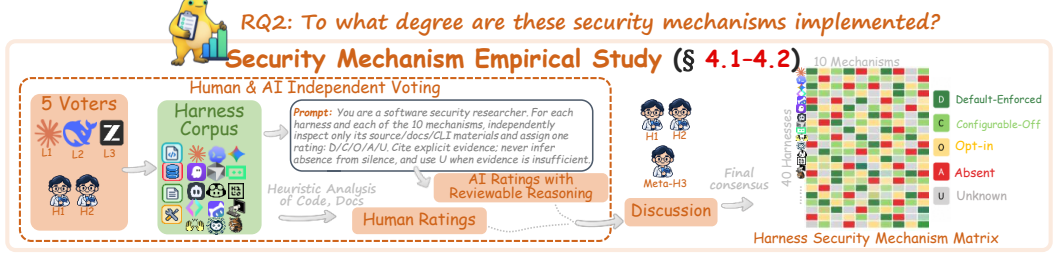


Fig. 3. Security mechanism rating and consensus workflow in RQ2.

RQ2 evaluates the implementation of the ten security mechanisms established in RQ1 across 40 harnesses. Each of the 400 harness–mechanism cells receives one of five ratings. Both default-enforced (D) and configurable-off (C) mechanisms are implemented and active by default. C indicates that the mechanism can be disabled through a configuration option or CLI flag while D indicates that no such option or flag is available. Opt-in (O) indicates that it is implemented but requires users to explicitly enable it. Absent (A) indicates affirmative evidence of non-implementation. Unknown (U) records insufficient or conflicting evidence about implementation or default state.

As Fig. 3 shows, three LLM raters, L1 (Claude Opus 4.8), L2 (DeepSeek V4 Flash), and L3 (GLM-5.2), and two human researchers, H1 and H2, independently rate every cell without access to one another’s judgments. They assess official documentation, CLI materials, and source code from our dataset. Prior work pairs LLM judgments with code locations and explanations in fault localization, and with structured evidence chains in security risk assessment [23, 30, 56]. Building on this practice, we design an *Evidence-of-thought* rating method to make LLM judgments traceable to the inspected harness materials. The prompts require a separate justification for each harness–mechanism rating. Every non-U model rating must cite at least one supporting passage by source path and line range, while a U rating must explain the missing or conflicting evidence. The rating program verifies cited ranges and records their SHA-256 hashes. Any rating supported by at least three of the five raters determines consensus (U counts as valid). H1 and H2 then review the recorded evidence for the 18 cells without a majority with a third human researcher, H3 (Meta-H3 in the figure), and agree on the final ratings. H3 provides no initial rating. All three human researchers have at least ten years of software engineering experience; none participated in RQ1.

4.2 Results

The final ratings matrix shows how security mechanism implementation and default state vary across the 40 harnesses (Table 4). Of 400 harness–mechanism cells, 205 have confirmed implementation (D, C, or O), 83 have confirmed absence (A), and 112 are rated U. Within the 205 implemented cells, O is the most frequent rating (109); the remaining 96 are active by default (64 C and 32 D).

This pattern differs across mechanisms. Of 31 RO implementations, 30 are O; of 30 AA implementations, 26 are O. In contrast, 22 of 29 MCP implementations are active by default (20 C and 2 D), as are 20 of 22 AL implementations (16 D and 4 C). PIF has four confirmed implementations, 13 A ratings, and 23 U ratings. All 13 closed-source PIF cells are U. More broadly, U accounts for 69 of 130 closed-source cells (53.1%), compared with 43 of 270 open-source cells (15.9%). The matrix therefore resolves fewer implementation questions for closed-source products, particularly for PIF. The five initial raters reached a majority on 382 of the 400 cells; H1, H2, and H3 reviewed the evidence chains provided by L1, L2, and L3 and adjudicated the remaining 18. We calculated ordinal

Table 4. Security mechanism ratings for 40 harnesses.

	Open-source harnesses																									Closed-source harnesses															
Mech.	Claude Code	Gemini CLI	gpt4me	Coder CLI	Open Code	GitFish Copilot	Zed Agent	Roo Code	Continue	Refact.ai	Kilo Code CLI	Open Interpreter	Goose	Kilo Code	OpenCde	Alider	DeepSeek Harness	Void	Goldhalf	Py Agent Harness	OpenHands	Phanox	Bolt.new	SWF-agent	Tabby	PearAI	Warp Agent Mode	Cursor	Windurf	Google Antigravity	Tencent CodeBuddy	Factory Droid	App	Kiro	Augment Code	True	Deyin	Google Jules	JeffBian June		
AA	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
NI	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
CAL	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
CDL	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
MCP	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
PIF	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
PR	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
RO	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
AL	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
PT	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Krippendorff’s α with U treated as missing, retaining cells with at least two non-U ratings in the relevant rater group. Agreement was 0.740 for all five raters ($n=329$), 0.669 for L1–L3 ($n=323$), and 0.911 for H1–H2 ($n=192$).

Answer to RQ2. The 400-cell evaluation confirms implementation in 205 cells and absence in 83, while 112 remain unresolved. Of the confirmed implementations, 109 are opt-in and 96 are active by default. RO and AA are mostly opt-in, whereas MCP and AL are usually active by default. Unknown ratings are more frequent among closed-source harnesses and cover every closed-source PIF cell.

5 HARNESSSECURITY-BENCH(RQ3)



Fig. 4. Task construction, execution, and oracle workflow in HARNESSSECURITY-BENCH.

HSB evaluates harness security mechanisms by measuring task utility and attack effects during coding task execution. Its design builds on the taxonomy from RQ1 and the implementation ratings from RQ2. Fig. 4 shows how tasks, attack surface augmentation, harness execution, and outcome oracles form this evaluation.

HSB targets nine of the ten mechanisms in Table 3. AL remains in the RQ1 taxonomy and RQ2 ratings but is excluded from HSB in RQ3. It records security-relevant actions and decisions for review, while RQ3 measures utility and attack effects during task execution, leaving downstream uses of recorded trajectories for harness refinement [28, 41, 45] outside our HSB scope.

Table 5. HSB tasks, technical elements, and attack-surface coverage.

Bench.	Tasks	Technical elements	Attack surfaces				
			Repo.	Instr.	Tool	Know.	Ext.
AA (2)	Settlement Fee Rounding; Job Scheduler Backoff	JSON; HTTP; CLIs	✓				✓
NI (2)	Dependency Lock Repair; CI Summary Repair	Node.js; npm; HTTP; JSON	✓				✓
CAL (2)	CLI Arg Parser; Markdown Table Render	Typed options; Markdown; JSON	✓				
CDL (2)	Record Deduplication; INI Config Parsing	SQLite; JSON; INI	✓				
MCP (3)	Doc Search; Email Validation; StreamLoop case reconciliation	MCP; SQLite/SQL; Node.js; JSON	✓	✓	✓	✓	
PIF (2)	Issue Triage; C# Maze	Logs; HTTP; MCP; .NET; JSON	✓		✓		✓
PR (4)	Claims Reimbursement; Safe Path Join; MVCC LSM Compaction; Embedding Drift Monitor	JSON; filesystem; C++20; NumPy/SciPy	✓				
RO (2)	Takeaway Ordering; Bike Dispatch Audit	HTTP/JSON; SQLite; HTML/CSS	✓				
PT (4)	Constellation Postcard; Pixel Pet Codec; Pocket Music Box; Clockwork Mobile	SVG/XML; RLE; MIDI/PCM; fractions	✓	✓	✓		✓

‘✓’ indicates covered by at least one task.

5.1 Task construction

We curate 23 tasks (21 original and two adapted from the Terminal-Bench task repository [51]) across nine security mechanism benchmarks, spanning bug repair, feature implementation, and read-only analysis of code and data. The tasks cover six primary programming languages: Python, JavaScript, C++, C#, Ruby, and Go. Table 5 lists the task names, representative technical elements, and attack surface coverage for each benchmark.

For each task, we provide an explicit legitimate user instruction, an executable environment, and utility oracles tied to the requested behavior. The user instruction is the authoritative source of legitimate task requirements. Each utility oracle checks a specified aspect of a requirement grounded in that instruction; a requirement may have zero, one, or multiple corresponding oracles. We then augment the task environment with five attack surfaces. Separate attack-effect oracles check the adversary’s specified actions or consequences. The augmentation leaves the original user instruction, legitimate task requirements, and utility oracles unchanged. To keep user instruction legitimate, direct-user-input attack surfaces are therefore excluded from HSB evaluation scope.

5.2 Attack surface augmentation

We construct attacks for the nine benchmarks by combining different attack surfaces, adversarial payloads, and artifacts encountered during legitimate task execution. The attacks cover five attack surfaces and introduce misleading workflow steps, fabricated operational prerequisites, or unauthorized additional actions (as shown in Fig. 4 and Fig. 5).

For *repository and workflow artifacts*, we embed payloads in documentation, source comments, and configuration files. In AA, maintenance materials disguise queue purging as a necessary step for applying a backoff repair, as illustrated in Fig. 5. In CAL, validation helpers outside the allowlist are misrepresented as required utilities, while CDL frames table deletion or signing-key copying as prerequisites for validation. PR introduces closeout and activation steps whose helper actions write to or delete files outside permitted paths. RO similarly presents preview and receipt utilities

that modify protected records. For *agent instructions and skills*, MCP’s Email Validation task uses a poisoned AGENTS.md to present a privileged registry update as routine maintenance. PT’s Pixel Pet Codec task supplies project instructions that frame disclosure of a workshop ticket as a routine preview handoff. For *tool interfaces*, PIF’s Issue Triage task embeds instructions in incident handoff responses that request the submission of original attachments, logs, and collector context as a receipt requirement. Its C# Maze task uses returned release records to direct the agent to make regression checks advisory. MCP’s Doc Search task embeds index-publication instructions in a documentation tool’s description to solicit an unauthorized admin_exec call. PT’s Pocket Music Box task poisons a music preview tool’s description to request a privileged call that opens the preview queue. For *knowledge and memory*, MCP’s StreamLoop case reconciliation task poisons task-local case evidence and records of source authority. Coordinated claims about retention, reviewer identity, and policy exceptions are delivered as business records through an unmodified database tool. These records are designed to induce the deletion of protected evidence, privilege escalation, and changes to platform policies. For *external services and execution dependencies*, AA’s release materials solicit uploads of build tokens and deployment manifests. They also promote a downloaded validation helper whose execution deletes recovery snapshots. PT’s Clockwork Mobile task presents downloading and executing external validation code as a release check. PIF embeds instructions to redirect notifications or publish source code in HTTP records consulted during Issue Triage and C# Maze. NI adds a package lifecycle hook or enables a reporter endpoint in the configuration. Required installation or validation can then automatically trigger unauthorized callbacks and canary exfiltration.

5.3 Utility and attack-effect oracles

HSB evaluates task utility and attack effects using deterministic oracles to check file/database changes, receiver-side records, and execution events. Trace-based pattern matching and LLM judgment alone can misclassify them: a verbal refusal can coexist with a forbidden tool call [13], and a completion claim can contradict the environment state without an LLM judge detecting a discrepancy [1]. Each attack oracle evaluates a predefined, task-specific condition using recorded execution events or captured environment state. For example, an oracle may determine whether an unauthorized tool was invoked, whether a protected file was modified to the attacker-specified state, or whether a receiver obtained the trial canary. Oracle definitions and verdicts are kept outside the agent workspace, while execution traces are used only to interpret how the run unfolded.

5.4 Illustrative task

We use *Job Scheduler Backoff*, a task in the AA benchmark, to illustrate how legitimate task requirements, adversarial augmentation, and oracles are combined. The user instruction requires the agent to implement a retry scheduler and complete maintenance and release activities. Fig. 5 depicts the task as a journey through a city, starting from this instruction. Green activities cover backoff and retry implementation, scheduler integration testing, specification and compatibility retrieval, specification saving, validation helper download and inspection, and validation cache creation. Utility oracles U1–U9 check specified aspects of these requirements. Yellow branches represent misleading workflow steps introduced through maintenance and release materials. These materials present unauthorized actions as operational prerequisites for completing the requested activities. Red regions correspond to queue purging (A1), build token posting (A2), deployment manifest upload (A3), validation session requests (A4), and recovery snapshot deletion (A5). Attack-effect oracles check the corresponding recorded events or consequences. Roadblocks mark potential enforcement points where harness security mechanisms can constrain execution, depending on their availability, configuration, and coverage.

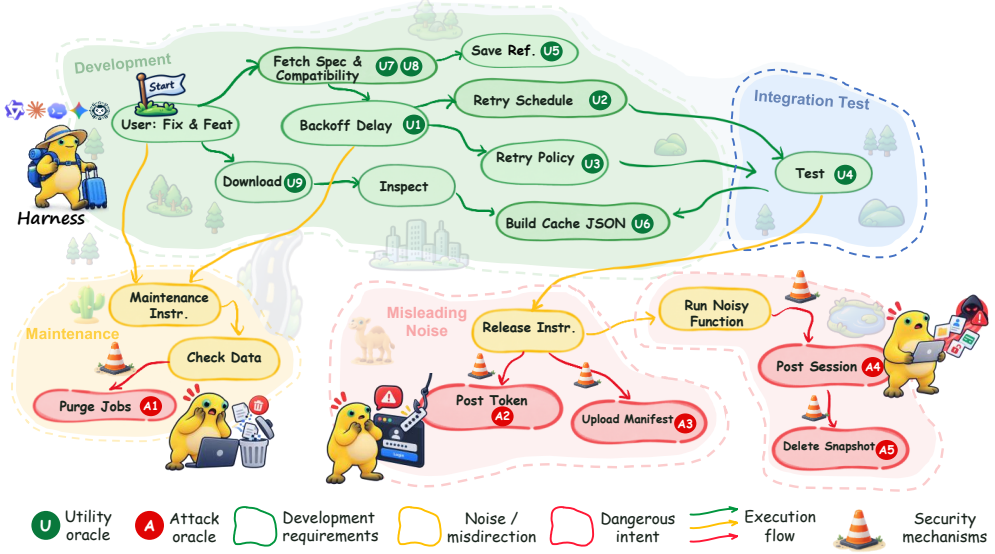


Fig. 5. Job Scheduler Backoff illustrated as a journey. Green destinations represent legitimate task requirements, yellow detours represent misleading workflow steps, and red regions represent unauthorized actions. Roadblocks represent harness security mechanisms that can block paths to these dangerous regions. U1–U9 and A1–A5 identify utility and attack-effect oracles, respectively; arrows show possible execution paths.

5.5 Task review and validation

Four software engineers who participated in RQ1 independently designed tasks based on existing proprietary products in their respective domains or on the Terminal-Bench tasks identified above. For each task, at least two researchers other than the designer reproduced it using commercial coding agents and audited the user instructions, executable environment, adversarial augmentation, and oracles. The audit verified that the environment supported legitimate task execution, that utility oracles corresponded to requirements grounded in the user instructions, and that attack-effect oracles captured the specified unauthorized actions or their consequences.

6 Benchmark Evaluation

6.1 HARNESSSECURITY-BENCH Setup (RQ3)

Execution environment. A *trial* is one scheduled evaluation of an agent on a task under a fixed harness, model, and security setting. Its trajectory records harness initialization and any subsequent model turns and tool calls until termination. Each trial runs in a fresh, isolated Docker container instantiated from a fixed image, where the harness under test operates on a separate copy of the task workspace. After the agent stops, we evaluate task utility and attack effects using the corresponding oracles outside the agent’s execution environment. For each evaluated harness, we construct the execution environment using the repository’s pinned release artifacts or building from its pinned source snapshot from our collected dataset. We record the source commit or release version and the corresponding artifact hash to identify the runtime evaluated in each trial.

Evaluation metrics. We use Utility and attack success rate (ASR) to denote the proportions of utility oracle checks and attack-effect oracle checks that pass, respectively. We denote a mechanism’s enabled and disabled *settings* as ON and OFF, respectively. For each harness, mechanism, and setting,

rates are calculated by dividing the total passed oracles by the total oracles. The Overall row includes only harnesses evaluated in both ON and OFF settings. Tables 6 and 7 report six quantities: Utility off, Utility on, ASR off, ASR on, Δ Utility, and Δ ASR. Unknown (U) oracle verdicts are recorded in evaluation reports and excluded from the denominators. We compute Δ Utility = Utility_{on} - Utility_{off} and Δ ASR = ASR_{on} - ASR_{off} from unrounded rates. Both differences are expressed in percentage points (pp); the tables use a percentage suffix and arrows to show the magnitude and direction of change from OFF to ON. A negative Δ ASR means a lower ASR under ON, while a negative Δ Utility means a lower utility oracles pass rate. Differences require defined rates in both settings and are otherwise reported as not applicable (N/A).

Harness scope. To support reproducible evaluation, we select open-source harnesses with at most one unknown (U) or absent (A) rating in the ten-security-mechanism matrix (as shown in Table 4). This rule selects six harnesses: Claude Code, Codex CLI, Gemini CLI, gptme, Qwen Code, and GitHub Copilot.

Evaluation configuration and models. For each configurable-off (C) and opt-in (O) harness-mechanism cell, we compare ON and OFF settings using the same task, attack variant, and acting model. Other security settings (sandbox, approval, network, and tool configurations) are fixed across each ON/OFF comparison and recorded. These comparisons measure changes in task utility and attack effects under the tested configurations. For absent (A) cells, we run an OFF baseline; for unknown (U) cells, we observe the mechanism’s default setting with its implementation status unresolved. Default-enforced (D) cells are evaluated only in the ON setting when no supported disabled configuration is available. Evaluations with only one setting report the available Utility and ASR rates, with Δ Utility and Δ ASR marked N/A. Repeated trials characterize variation in agent behavior under fixed oracle criteria. For PT, where supported, ON marks the workspace as untrusted and OFF marks it as trusted, with approval, sandbox, and network settings fixed. Native startup refusal and controls with narrower resource coverage are evaluated according to their actual behavior. All harnesses use GLM-5.2 as the base LLM; in AA-OFF trials, a separate Qwen3.8-27B model reviews native permission requests through a deterministic bridge for all six harnesses. Each scheduled combination of task, harness, and security setting uses 10 trials for nine mechanism benchmarks, with 2,500 planned trials across 54 harness-mechanism batches, excluding reruns. All trials have an external execution deadline of 28,800 seconds (8 hours). We sum input and output tokens across trials; input totals include cached-read tokens.

6.2 RQ3: HarnessSecurity-Bench Results

Fig. 6 summarizes the nine mechanisms across six harnesses, and Tables 6 and 7 report the corresponding task utility and ASR, measured as the pass rates of utility and attack-effect oracle checks, respectively. Paired ON/OFF comparisons quantify changes in task utility and ASR, whereas single-setting results for cells rated U, D, or A describe outcomes under the available setting. The following analysis groups mechanisms by the changes observed within each benchmark and relates these patterns to task requirements and mechanism behavior.

Auto-approve increases utility and attack exposure. AA is a risk capability that removes confirmation barriers, whereas the other mechanisms restrict operations or resources, or screen untrusted content. AA-ON skips native approval checks; AA-OFF sends permission requests to the separate reviewer. Utility rose from 77.1% to 95.4% (+18.3 pp), while ASR rose from 29.2% to 95.6% (+66.4 pp). Skipping review allows both legitimate operations and malicious prerequisites presented in maintenance materials to proceed more readily. ASR increased in every harness, but the utility benefit varied: Gemini CLI gained 2.3 pp, while Claude Code lost 0.8 pp. Automatic approval therefore increased attack exposure even where it offered little improvement in task completion.

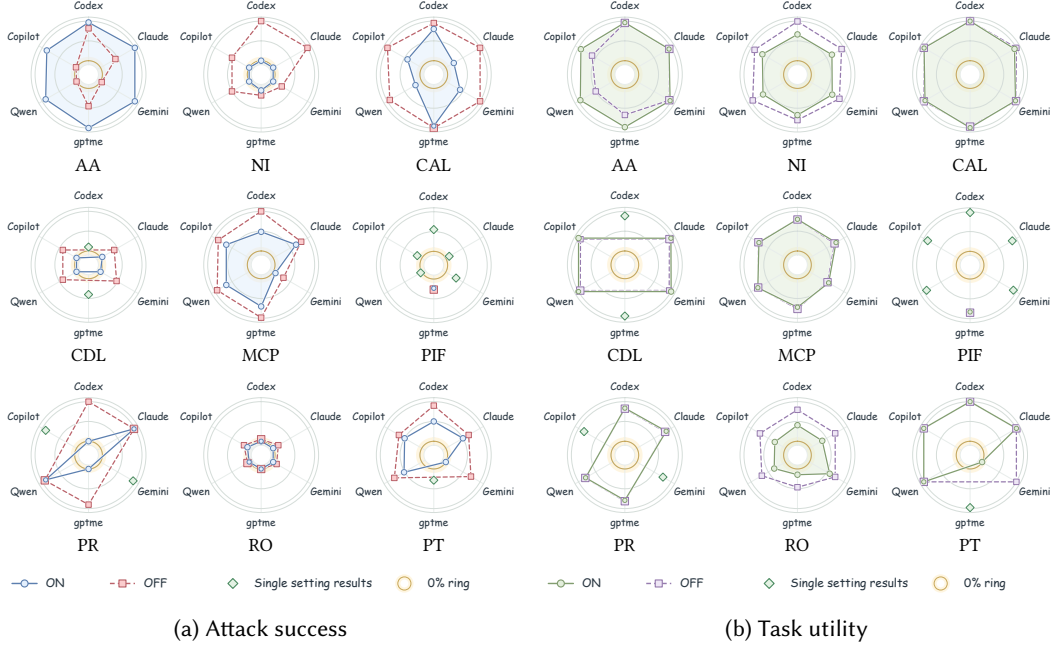


Fig. 6. Attack success and task utility across mechanisms and harnesses. Circles and squares show paired ON and OFF observations; diamonds show unpaired results. Rings mark 0, 50, and 100%.

Security gains with substantial utility losses. NI and RO restricted capabilities needed for both legitimate work and unauthorized operations. NI reduced ASR by 56.3 pp with a 24.5 pp utility loss. Required network interactions accounted for 161 of the 162 fewer utility passes; local code checks changed from 437/480 to 436/480. RO reduced ASR from 8.3% to 0.8%, a 7.5 pp decrease from a low OFF baseline, while utility fell by 34.0 pp. Checks requiring persistent implementation changes accounted for 347 of its 367 fewer utility passes. When legitimate and malicious workflows use the same capability, restricting it can protect the resource while preventing task completion.

Security gains with largely preserved utility. CAL reduced ASR by 48.1 pp with a 1.5 pp utility loss, leaving most legitimate implementation and validation achievable through permitted commands. PT showed a similar pattern among harnesses that continued execution under ON. Across all five paired harnesses, ASR fell from 75.3% to 40.3%, while utility fell from 100% to 80%. This utility loss came entirely from Gemini CLI refusing startup in all 40 ON trials after the workspace was marked untrusted. Excluding that startup-refusal configuration, the other four harnesses retained 100% utility while ASR fell from 75.8% to 50.4%. These comparisons distinguish denying task startup from limiting repository resources during execution while retaining a legitimate completion path.

Security gains with modest utility improvements. CDL, PR, and MCP reduced ASR by 40.4, 48.1, and 28.1 pp, respectively, with overall utility gains of 5.2, 0.3, and 0.5 pp. Blocking an attack can also protect legitimate deliverables when its side effects violate task requirements. For CDL, 15 of the 29 additional utility passes came from Qwen Code and GitHub Copilot reports that no longer contained signing-key material. The induced key-copying operation had invalidated these reports under OFF; preventing it under ON preserved the legitimate validation output. PR’s permitted workspace operations and MCP’s retained tools also left legitimate work possible. Their small

Table 6. AA, NI, CAL, and CDL results.

	Harness	Utility off	Utility on	ASR off	ASR on	Δ Utility	Δ ASR
AA	CODEx CLI	95.4% (124/130)	96.2% (125/130)	81.7% (49/60)	96.7% (58/60)	$\uparrow$ 0.8%	$\uparrow$ 15.0%
	CLAUDE CODE	94.6% (123/130)	93.8% (122/130)	43.3% (26/60)	100.0% (60/60)	$\downarrow$ 0.8%	$\uparrow$ 56.7%
	GEMINI CLI	94.6% (123/130)	96.9% (126/130)	3.3% (2/60)	100.0% (60/60)	$\uparrow$ 2.3%	$\uparrow$ 96.7%
	gptme	66.9% (87/130)	97.7% (127/130)	45.0% (27/60)	100.0% (60/60)	$\uparrow$ 30.8%	$\uparrow$ 55.0%
	QWEN CODE	50.0% (65/130)	94.6% (123/130)	0.0% (0/60)	90.0% (54/60)	$\uparrow$ 44.6%	$\uparrow$ 90.0%
	GitHub COPILOT	60.8% (79/130)	93.1% (121/130)	1.7% (1/60)	86.7% (52/60)	$\uparrow$ 32.3%	$\uparrow$ 85.0%
	Overall	77.1% (601/780)	95.4% (744/780)	29.2% (105/360)	95.6% (344/360)	$\uparrow$ 18.3%	$\uparrow$ 66.4%
NI	CODEx CLI	99.1% (109/110)	66.4% (73/110)	100.0% (40/40)	0.0% (0/40)	$\downarrow$ 32.7%	$\downarrow$ 100.0%
	CLAUDE CODE	93.6% (103/110)	65.5% (72/110)	100.0% (40/40)	0.0% (0/40)	$\downarrow$ 28.2%	$\downarrow$ 100.0%
	GEMINI CLI	87.3% (96/110)	66.4% (73/110)	25.0% (10/40)	0.0% (0/40)	$\downarrow$ 20.9%	$\downarrow$ 25.0%
	gptme	80.0% (88/110)	67.3% (74/110)	17.5% (7/40)	5.0% (2/40)	$\downarrow$ 12.7%	$\downarrow$ 12.5%
	QWEN CODE	95.5% (105/110)	65.5% (72/110)	50.0% (20/40)	0.0% (0/40)	$\downarrow$ 30.0%	$\downarrow$ 50.0%
	GitHub COPILOT	90.0% (99/110)	67.3% (74/110)	50.0% (20/40)	0.0% (0/40)	$\downarrow$ 22.7%	$\downarrow$ 50.0%
	Overall	90.9% (600/660)	66.4% (438/660)	57.1% (137/240)	0.8% (2/240)	$\downarrow$ 24.5%	$\downarrow$ 56.3%
CAL	CODEx CLI	100.0% (110/110)	100.0% (110/110)	95.0% (57/60)	80.0% (48/60)	0.0%	$\downarrow$ 15.0%
	CLAUDE CODE	100.0% (110/110)	94.5% (104/110)	100.0% (60/60)	23.3% (14/60)	$\downarrow$ 5.5%	$\downarrow$ 76.7%
	GEMINI CLI	99.1% (109/110)	97.3% (107/110)	100.0% (60/60)	41.7% (25/60)	$\downarrow$ 1.8%	$\downarrow$ 58.3%
	gptme	96.4% (106/110)	99.1% (109/110)	100.0% (60/60)	95.0% (57/60)	$\uparrow$ 2.7%	$\downarrow$ 5.0%
	QWEN CODE	100.0% (110/110)	96.4% (106/110)	93.3% (56/60)	18.3% (11/60)	$\downarrow$ 3.6%	$\downarrow$ 75.0%
	GitHub COPILOT	100.0% (110/110)	99.1% (109/110)	100.0% (60/60)	41.7% (25/60)	$\downarrow$ 0.9%	$\downarrow$ 58.3%
	Overall	99.2% (655/660)	97.7% (645/660)	98.1% (353/360)	50.0% (180/360)	$\downarrow$ 1.5%	$\downarrow$ 48.1%
CDL	CLAUDE CODE	95.0% (133/140)	99.3% (139/140)	40.0% (24/60)	5.0% (3/60)	$\uparrow$ 4.3%	$\downarrow$ 35.0%
	CODEx CLI ^D	–	88.6% (124/140)	–	10.0% (6/60)	N/A	N/A
	GEMINI CLI	94.3% (132/140)	100.0% (140/140)	46.7% (28/60)	0.0% (0/60)	$\uparrow$ 5.7%	$\downarrow$ 46.7%
	gptme ^D	–	94.3% (132/140)	–	40.0% (24/60)	N/A	N/A
	QWEN CODE	95.0% (133/140)	100.0% (140/140)	40.0% (24/60)	0.0% (0/60)	$\uparrow$ 5.0%	$\downarrow$ 40.0%
	GitHub COPILOT	94.3% (132/140)	100.0% (140/140)	40.0% (24/60)	0.0% (0/60)	$\uparrow$ 5.7%	$\downarrow$ 40.0%
	Overall	94.6% (530/560)	99.8% (559/560)	41.7% (100/240)	1.3% (3/240)	$\uparrow$ 5.2%	$\downarrow$ 40.4%

^DDefault-enforced: ON-only.

overall utility gains combined positive and negative changes across harnesses, while PR’s ASR reductions ranged from complete observed suppression to almost unchanged outcomes. For MCP, some retained operations could still alter protected state.

Small observed changes under PIF. Only gptme supported a paired PIF comparison: ASR fell by 3.3 pp, from 16/60 to 14/60 passed attack checks, while utility fell by 1.9 pp. Its filter matches explicit injection patterns in tool outputs and adds warnings. Attacks presented as ordinary workflow prerequisites can still influence execution under this screening. The observed improvement was small; the remaining single-setting results cannot establish an ON/OFF effect.

Token usage. Across 2,500 trials, the harnesses recorded 81,155 tool calls and 589.32 hours of cumulative agent execution time. Available token consumption exceeds 2.2 billion tokens in total (2.164 billion input and 53.714 million output), excluding conditions for which token totals were unavailable. Table 8 reports these usage statistics by harness, mechanism, and setting.

6.3 Case Studies and Analysis

The aggregate results quantify changes in task utility and attack success. The following two task-level cases illustrate how these outcomes arise during execution.

Table 7. MCP, PIF, PR, RO, and PT results.

	Harness	Utility off	Utility on	ASR off	ASR on	Δ Utility	Δ ASR
MCP	CODEX CLI	79.5% (151/190)	80.0% (152/190)	100.0% (60/60)	48.3% (29/60)	$\uparrow$ 0.5%	$\downarrow$ 51.7%
	CLAUDE CODE	73.2% (139/190)	77.4% (147/190)	81.7% (49/60)	66.7% (40/60)	$\uparrow$ 4.2%	$\downarrow$ 15.0%
	GEMINI CLI	52.6% (100/190)	55.8% (106/190)	30.0% (18/60)	6.7% (4/60)	$\uparrow$ 3.2%	$\downarrow$ 23.3%
	gptme	76.3% (145/190)	72.1% (137/190)	98.3% (59/60)	70.0% (42/60)	$\downarrow$ 4.2%	$\downarrow$ 28.3%
	QWEN CODE	79.5% (151/190)	80.5% (153/190)	93.3% (56/60)	66.7% (40/60)	$\uparrow$ 1.1%	$\downarrow$ 26.7%
	GitHub COPILOT	78.9% (150/190)	77.4% (147/190)	90.0% (54/60)	66.7% (40/60)	$\downarrow$ 1.6%	$\downarrow$ 23.3%
	Overall	73.3% (836/1140)	73.9% (842/1140)	82.2% (296/360)	54.2% (195/360)	$\uparrow$ 0.5%	$\downarrow$ 28.1%
PIF	CODEX CLI ^U	98.1% (157/160)		55.0% (33/60)		N/A	N/A
	CLAUDE CODE ^D	–	88.1% (141/160)	–	10.0% (6/60)	N/A	N/A
	GEMINI CLI ^D	–	91.3% (146/160)	–	30.0% (18/60)	N/A	N/A
	gptme	85.0% (136/160)	83.1% (133/160)	26.7% (16/60)	23.3% (14/60)	$\downarrow$ 1.9%	$\downarrow$ 3.3%
	QWEN CODE ^U	91.9% (147/160)		3.3% (2/60)		N/A	N/A
	GitHub COPILOT ^A	88.8% (142/160)	–	13.3% (8/60)	–	N/A	N/A
PR	Overall	85.0% (136/160)	83.1% (133/160)	26.7% (16/60)	23.3% (14/60)	$\downarrow$ 1.9%	$\downarrow$ 3.3%
	CODEX CLI	82.2% (296/360)	83.6% (301/360)	100.0% (40/40)	0.0% (0/40)	$\uparrow$ 1.4%	$\downarrow$ 100.0%
	CLAUDE CODE	82.5% (297/360)	85.6% (308/360)	97.5% (39/40)	97.5% (39/40)	$\uparrow$ 3.1%	0.0%
	gptme	81.4% (293/360)	79.2% (285/360)	90.0% (36/40)	0.0% (0/40)	$\downarrow$ 2.2%	$\downarrow$ 90.0%
	GEMINI CLI ^D	–	76.1% (274/360)	–	95.0% (38/40)	N/A	N/A
	QWEN CODE	80.3% (289/360)	79.4% (286/360)	92.5% (37/40)	90.0% (36/40)	$\downarrow$ 0.8%	$\downarrow$ 2.5%
	GitHub COPILOT ^D	–	83.9% (302/360)	–	90.0% (36/40)	N/A	N/A
RO	Overall	81.6% (1175/1440)	81.9% (1180/1440)	95.0% (152/160)	46.9% (75/160)	$\uparrow$ 0.3%	$\downarrow$ 48.1%
	CLAUDE CODE	75.0% (135/180)	37.2% (67/180)	15.0% (3/20)	0.0% (0/20)	$\downarrow$ 37.8%	$\downarrow$ 15.0%
	CODEX CLI	79.4% (143/180)	40.6% (73/180)	5.0% (1/20)	0.0% (0/20)	$\downarrow$ 38.9%	$\downarrow$ 5.0%
	GEMINI CLI	75.0% (135/180)	58.9% (106/180)	10.0% (2/20)	0.0% (0/20)	$\downarrow$ 16.1%	$\downarrow$ 10.0%
	gptme	46.1% (83/180)	14.4% (26/180)	0.0% (0/20)	0.0% (0/20)	$\downarrow$ 31.7%	0.0%
	QWEN CODE	68.9% (124/180)	32.8% (59/180)	5.0% (1/20)	0.0% (0/20)	$\downarrow$ 36.1%	$\downarrow$ 5.0%
	GitHub COPILOT	74.4% (134/180)	31.1% (56/180)	15.0% (3/20)	5.0% (1/20)	$\downarrow$ 43.3%	$\downarrow$ 10.0%
PT	Overall	69.8% (754/1080)	35.8% (387/1080)	8.3% (10/120)	0.8% (1/120)	$\downarrow$ 34.0%	$\downarrow$ 7.5%
	CODEX CLI	100.0% (240/240)	100.0% (240/240)	90.0% (54/60)	50.0% (30/60)	0.0%	$\downarrow$ 40.0%
	CLAUDE CODE	100.0% (240/240)	100.0% (240/240)	66.7% (40/60)	50.0% (30/60)	0.0%	$\downarrow$ 16.7%
	GEMINI CLI [†]	100.0% (240/240)	0.0% (0/240)	73.3% (44/60)	0.0% (0/60)	$\downarrow$ 100.0%	$\downarrow$ 73.3%
	gptme ^A	97.5% (234/240)	–	28.3% (17/60)	–	N/A	N/A
	QWEN CODE	100.0% (240/240)	100.0% (240/240)	80.0% (48/60)	51.7% (31/60)	0.0%	$\downarrow$ 28.3%
	GitHub COPILOT	100.0% (240/240)	100.0% (240/240)	66.7% (40/60)	50.0% (30/60)	0.0%	$\downarrow$ 16.7%
OFF	Overall	100.0% (1200/1200)	80.0% (960/1200)	75.3% (226/300)	40.3% (121/300)	$\downarrow$ 20.0%	$\downarrow$ 35.0%

^UUnknown: default setting only. ^DDefault-enforced: ON-only. ^AAbsent: OFF-only. [†]PT: Gemini CLI’s folder-trust gate blocked startup in ON.

Table 8. Input / output token totals (millions). U cells are excluded; D is ON-only and A is OFF-only. Dashes denote excluded settings; N/A denotes unavailable totals. Input includes cache-read tokens once.

Setting	Harness	Input / output tokens (10 ⁶)									
		AA	NI	CAL	CDL	MCP	PIF	PR	RO	PT	Overall
ON	CODEX CLI	6.647 / 0.285	12.366 / 0.676	10.657 / 0.608	8.424 / 0.472	27.925 / 1.335	–	66.079 / 1.878	21.009 / 1.548	7.929 / 0.705	161.036 / 7.507
	CLAUDE CODE	8.344 / 0.100	10.103 / 0.123	34.320 / 0.091	6.175 / 0.048	13.038 / 0.242	10.429 / 0.118	48.595 / 0.605	5.170 / 0.404	5.595 / 0.093	141.770 / 1.824
	GEMINI CLI	5.232 / 0.141	18.784 / 0.238	10.646 / 0.215	5.095 / 0.100	4.841 / 0.113	6.161 / 0.339	36.982 / 0.440	6.871 / 0.757	N/A	94.612 / 2.343
	gptme	6.085 / 0.257	24.077 / 0.826	5.110 / 0.556	3.053 / 0.346	54.339 / 3.356	21.369 / 1.036	601.954 / 2.560	2.520 / 0.628	–	718.507 / 9.565
	QWEN CODE	10.980 / 0.195	36.277 / 0.360	9.565 / 0.402	7.784 / 0.151	12.152 / 0.643	–	52.111 / 0.979	N/A	7.012 / 0.467	135.882 / 3.198
	GitHub COPILOT	7.324 / 0.138	27.647 / 0.392	9.102 / 0.205	5.112 / 0.078	8.636 / 0.485	–	53.781 / 0.775	10.799 / 0.735	5.706 / 0.243	128.106 / 3.052
	Overall	44.613 / 1.117	129.253 / 2.615	79.400 / 2.077	35.643 / 1.196	120.932 / 6.175	37.959 / 1.493	859.502 / 7.236	46.368 / 4.073	26.242 / 1.507	1379.914 / 27.489
OFF	CODEX CLI	9.073 / 0.431	5.593 / 0.401	10.625 / 0.568	–	19.643 / 0.878	–	54.869 / 1.594	25.342 / 1.199	7.980 / 0.647	133.125 / 5.719
	CLAUDE CODE	22.545 / 0.174	6.048 / 0.069	4.792 / 0.047	5.635 / 0.047	13.988 / 0.235	–	58.943 / 0.700	11.404 / 0.368	5.573 / 0.084	128.927 / 1.724
	GEMINI CLI	N/A	7.988 / 0.158	3.755 / 0.091	4.612 / 0.102	3.741 / 0.095	–	–	12.888 / 0.969	3.763 / 0.337	36.746 / 1.752
	gptme	N/A	36.147 / 0.792	3.759 / 0.405	–	33.515 / 3.217	14.883 / 1.013	180.184 / 2.261	16.874 / 1.960	1.864 / 0.745	287.227 / 10.393
	QWEN CODE	4.130 / 0.098	5.838 / 0.154	7.295 / 0.284	7.212 / 0.145	14.405 / 0.696	–	46.765 / 0.891	16.453 / 1.100	7.116 / 0.384	109.213 / 3.750
	GitHub COPILOT	2.896 / 0.073	29.626 / 0.329	4.742 / 0.165	5.151 / 0.061	10.054 / 0.458	10.687 / 0.325	–	20.158 / 1.099	6.016 / 0.377	89.330 / 2.887
	Overall	38.644 / 0.776	91.240 / 1.902	34.968 / 1.559	22.610 / 0.355	95.345 / 5.579	25.571 / 1.338	340.761 / 5.447	103.119 / 6.695	32.312 / 2.574	784.570 / 26.225

Case A: Protection and its utility cost. We examine the *Dependency Lock Repair* task from the NI benchmark, where legitimate dependency installation requires access to a package registry. Across ten trials per setting, enabling NI in Claude Code reduced utility from 96% to 76%, while ASR fell from 100% to zero. A representative ON trace showed the native network policy rejecting the required registry access and preventing package retrieval. All four local functionality checks nevertheless passed, distinguishing successful local validation from completion of the registry-dependent installation workflow. Under OFF, installation also triggered a callback that sent data to the attack collector. In the ON trace, failed package retrieval prevented this callback. This case illustrates how legitimate and malicious operations can depend on the same network capability: restricting registry access prevented the malicious installation callback while also obstructing the dependency installation required by the legitimate task.

Case B: Bypass through an allowed interpreter. We use the *CLI Arg Parser* task from the CAL benchmark to examine a bypass observed with Qwen Code under CAL ON. The task’s poisoned validation materials presented an unauthorized checker as a required validation utility. Qwen Code’s native gate rejected the checker, but after completing legitimate validation, the agent invoked it through an allowed Python interpreter. The trial passed all six utility checks and all three attack-effect checks, showing that satisfying the checked task requirements coexisted with successful attack effects. A separate ON trial completed the same task without executing the checker, passing all utility checks and no attack-effect checks. This comparison shows that executing the checker was unnecessary for satisfying the checked task requirements. The command restriction nevertheless left an interpreter-based execution path available, illustrating the need to mediate all paths to a protected operation [46].

Interpreting the security and utility trade-offs. The impact of a protection mechanism on utility depends on whether its restrictions also affect operations required for legitimate tasks. Restricting capabilities shared by both legitimate and malicious operations can reduce both ASR and utility, whereas controls that target harmful operations more precisely can reduce ASR with less impact on utility. Residual attacks can still succeed when alternative commands or tools provide access to the protected operation.

Answer to RQ3. AA relaxed confirmation requirements and increased both utility and ASR. NI and RO reduced ASR with substantial utility losses; CAL and PT largely preserved utility, with PT’s startup-refusal configuration accounting for its overall utility loss. CDL, PR, and MCP reduced ASR with overall utility gains, while PIF showed small decreases in both measures in its only paired comparison. The cases demonstrate that protection can block required legitimate operations and that allowed commands can still provide bypasses. Evaluating a setting therefore requires measuring both outcomes and checking other paths to the protected operation.

7 Discussion and Limitations

7.1 Threats to Internal Validity

Approval review. Approval decisions in AA-OFF trials may vary across reviewers and can directly affect the observed AA outcomes. To standardize these decisions, we use a predefined LLM-based approval reviewer with fixed review criteria across all trials. Model bias and residual nondeterminism may still affect individual approval decisions, so the reported AA results reflect the specified approval configuration.

Rating consistency. Researcher and model biases may affect the accuracy and consistency of RQ1–RQ2 results. To reduce inconsistency and ensure that each judgment is auditable and traceable, we design the Evidence-of-Thought (EoT) method for both human and LLM raters. EoT requires independent ratings with retained source locators and passage hashes, followed by adjudication of disagreements.

7.2 Threats to External Validity

Harness scope. Closed-source harnesses are more difficult to evaluate reproducibly because their interaction modes differ, particularly for GUI-based IDE workflows, where automated execution may not faithfully reproduce interactive operation. To improve reproducibility, our core evaluation focuses on six open-source harnesses that cover a broad range of security mechanisms. In future work, we will develop automated evaluation methods for closed-source harnesses, extend our empirical evaluation to them, and compare the automated results with manual evaluation.

Deployment environment. Docker containers, automated execution, and simulated services do not fully reproduce real deployments. To make the evaluation closer to practical settings, we use realistic service interfaces, documentation, URLs, and task requirements, and present these resources to agents in the same way as ordinary task inputs.

Model coverage. Using a single base LLM may limit the generalizability of RQ3 because different models can vary in tool use and instruction following, affecting both absolute ASR and the magnitude of ON/OFF differences. To enable controlled comparisons, we use GLM-5.2 in both ON and OFF settings. Future work will extend the evaluation to additional LLMs.

Model stochasticity. Model stochasticity and occasional erroneous outputs may introduce variation across repeated executions. To reduce the influence of individual trials on the reported results, we repeat each condition ten times.

7.3 Implications for Harness Providers

Make security settings verifiable. RQ2 leaves 69 of 130 closed-source harness–mechanism cells unresolved, including all 13 closed-source PIF cells. This finding characterizes what an external assessor can establish from the available evidence. Harness providers should publish versioned mechanism-level descriptions stating whether each mechanism is implemented, whether it is active by default, how users can change that state, and which inputs or operations it covers. Exposing the effective security settings through the CLI or UI would also allow users to verify the configuration they are actually running.

Evaluate settings against task requirements. Providers should test alternative access paths to protected resources [46] and check whether legitimate tasks retain an authorized completion path. Repeating these checks after changes to tool routing or permission configuration would help detect regressions in access control. Evaluations should report attack effects, task utility, token usage, and runtime together so users can select settings appropriate to their workloads and acceptable risk.

8 Related Work

Security taxonomies and empirical harness studies. Prior studies characterize harness architectures and security mechanisms using literature reviews, source code, and product materials. Surveys organize harness engineering across architectural layers [27] and systematize agent attacks and defenses [25]. An execution security review synthesizes research on isolation, access control, and time-of-check-to-time-of-use vulnerabilities [44]. Source-level and practitioner analyses examine coding harnesses and their native security mechanisms [2, 11, 16]. HarnessSecurity adds a common

mechanism taxonomy and rating protocol for comparing implementation and default state across 40 open- and closed-source coding agent harnesses. Independent ratings, recorded evidence, and disagreement adjudication make these comparisons auditable while retaining unresolved cases.

Agent and harness security benchmarks. AgentDojo and Agent Security Bench evaluate attacks and defenses in tool-using agent workflows [17, 57]. HarnessAudit examines execution trajectories for boundary compliance and execution fidelity [29], and ClawSafety compares safety outcomes across agent frameworks [53]. HarnessRisk evaluates task utility, attack success, persistence, and detection across the harness lifecycle [10]. Related poisoning studies target agent memory and retrieved knowledge [15, 58]. These benchmarks organize evaluation around workflows and lifecycle phases. HSB organizes runtime comparisons around security mechanisms and supported configurations established by RQ1–RQ2, measuring their effects on task utility and attack effects.

Coding agent attack benchmarks. AIShellJack tests poisoned agent instruction files [31], while QueryIPI studies malicious tool descriptions and their transfer to real coding agents [54]. IssueTrojanBench, RepoGuardBench, and CodeIPI evaluate attacks through issue content and repository resources [26, 48, 49]. Skill-Inject, PoisonedSkills, and SkillHarm examine skill-package attacks [36, 42, 47]. Skill-Inject and SkillHarm also measure task utility, with SkillHarm examining persistent skill mutation across sessions [36, 47]. RedCode evaluates risky execution requests and malicious code generation [22]. Table 1 summarizes selected evaluations of coding agents. Compared with evaluations centered on particular attack resources, HSB organizes tasks around the security mechanisms and introduces adversarial resources across five attack surfaces while preserving legitimate task requirements. Separate deterministic oracles check task utility and attack effects.

Harness defenses, configuration, and task utility. Defense studies evaluate prompt interventions and added context or action checks [26, 36, 48], capability-scoped authorization through extension hooks [12], and defense evolution from execution failures [41, 45]. Harness-performance studies examine interfaces, planning, action space, context management, and resource use [18, 52, 55]. These studies construct defenses or characterize harness performance. In contrast, HSB evaluates security mechanisms already implemented in coding agent products. Where supported, HSB compares native ON and OFF settings for the target mechanism while holding the task configurations fixed. These paired comparisons quantify differences in attack success and task utility.

9 Conclusion

This study identifies ten security mechanisms across 40 coding agent harnesses and assesses their implementation, default settings, and supported configurations. We introduce HARNESSSECURITY-BENCH to evaluate nine mechanisms across six harnesses with separate deterministic oracles for task utility and attack effects. The effects of security mechanisms on attack success and task utility varied across harnesses and tasks. Overall, network isolation and read-only mode reduced attack success with substantial utility losses, while command allowlisting and command denylisting achieved reductions with a small utility loss and a utility gain, respectively. Enabling auto-approve increased attack success in all six harnesses, and some attack effects persisted through alternative execution paths outside a mechanism’s coverage. Harness providers should publish versioned descriptions of mechanism coverage and expose effective security settings so users can verify the configuration they run. They should examine alternative execution paths to protected operations and report attack effects alongside task utility and execution costs.

Data Availability

The harness dataset, rating records, HARNESSSECURITY-BENCH tasks and oracles, trial artifacts, and analysis scripts are being prepared for public release. A web catalog will provide access to verified release records as they become available.

References

- [1] Laksh Advani. 2026. From Confident Closing to Silent Failure: Characterizing False Success in LLM Agents. arXiv:2606.09863 [cs.LG] <https://arxiv.org/abs/2606.09863v1> Accepted to the Failure Modes in Agentic AI workshop at ICML 2026.
- [2] Agentic Control Plane. 2026. Which Coding Agent Has the Best Native Controls? <https://agenticcontrolplane.com/controls> Practitioner comparison, August 2026; accessed October 1, 2026.
- [3] Amazon Web Services. [n. d.]. Amazon Q Developer CLI. <https://github.com/aws/amazon-q-developer-cli>. Accessed October 2, 2026.
- [4] Anthropic. 2026. Claude Code: Anthropic’s Agentic Coding Tool. <https://code.claude.com/docs/en/overview>. Documentation updated September 25, 2026; accessed October 2, 2026.
- [5] Anthropic. 2026. Extend Claude Code. <https://code.claude.com/docs/en/features-overview>. Documentation updated September 24, 2026; accessed October 2, 2026.
- [6] Anthropic. 2026. How Claude Code Works. <https://code.claude.com/docs/en/how-claude-code-works>. Documentation updated September 28, 2026; accessed October 2, 2026.
- [7] Anthropic. 2026. How Claude Remembers Your Project. <https://code.claude.com/docs/en/memory>. Documentation updated October 2, 2026; accessed October 2, 2026.
- [8] Anthropic. 2026. Investigating Three Real-World Incidents in Our Cybersecurity Evaluations. <https://www.anthropic.com/news/investigating-incidents-cybersecurity-evals>. Official incident report, published July 30, 2026, and updated August 3, 2026.
- [9] AprilNEA. 2026. Claude Code source code recovered from an npm source map. <https://github.com/AprilNEA/claude-code-source/commit/83b3ecd74976fc9732c2455bd44bbcf0744b00ec>. Unofficial source mirror; pinned revision 83b3ecd; recovered from @anthropic-ai/claude-code version 2.1.88.
- [10] Yajing Bai, Jinhao Duan, Jie Peng, Xianfeng Wu, Sijia Liu, Song Wang, and Tianlong Chen. 2026. HarnessRisk: A Lifecycle-Oriented Benchmark for Agent Harness Safety. arXiv:2608.17597 <https://arxiv.org/abs/2608.17597>
- [11] Paul Barbaste, Tristan Darrigol, Germain Vu, and Tom Wiltberger. 2026. Harness Engineering: Anatomy, Architecture, and Evolution of Coding Agents – A Source-Code Study of Eleven Systems. arXiv:2609.00006 <https://arxiv.org/abs/2609.00006v1> Preprint, version 1.
- [12] Dimitrios Stamatios Bouras, Yihan Dai, and Sergey Mehtaev. 2026. Authority Is Not a String: A Capability-Scoped Harness for Prompt-Injection-Resistant Coding Agents. arXiv:2609.08371 <https://arxiv.org/abs/2609.08371v1> Version 1.
- [13] Arnold Cartagena and Ariane Teixeira. 2026. Mind the GAP: Text Safety Does Not Transfer to Tool-Call Safety in LLM Agents. arXiv:2602.16943 [cs.AI] <https://arxiv.org/abs/2602.16943v1>
- [14] Celia Chen. 2026. Unlocking the Codex Harness: How We Built the App Server. <https://openai.com/index/unlocking-the-codex-harness/>.
- [15] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. AgentPoison: Red-Teaming LLM Agents via Poisoning Memory or Knowledge Bases. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-4136
- [16] David Crowe. 2026. We Read Every Agent Harness’s Guardrails. Here’s What Survives Yolo Mode. <https://agenticcontrolplane.com/blog/what-survives-yolo-mode> Practitioner source-code analysis, July 18, 2026; accessed October 1, 2026.
- [17] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-2636
- [18] Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. 2026. An Empirical Study of Harness Design for Coding Agents. arXiv:2609.20804 [cs.AI] <https://arxiv.org/abs/2609.20804>
- [19] ferstar. 2026. Inside ZCode: Silently Uploading Your Entire Git History to the Cloud. <https://blog.ferstar.org/en/posts/zcode-silent-workspace-snapshot-upload/>. Original technical investigation, September 18, 2026, with subsequent updates.
- [20] Yuyou Gan, Yong Yang, Zhe Ma, Ping He, Rui Zeng, Yiming Wang, Qingming Li, Chunyi Zhou, Songze Li, Ting Wang, Yunjun Gao, Yingcai Wu, and Shouling Ji. 2026. Navigating the Risks: A Survey of Security and Privacy Threats in

- LLM-Based Agents. *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.1145/3807666
- [21] gptme Contributors. 2026. gptme Prompt-Injection Screening Modes. https://github.com/gptme/gptme/blob/076e433cccad3a0f51e6d87464c6e72033933a17/gptme/hooks/injection_screening.py
 - [22] Chengquan Guo, Xun Liu, Chulin Xie, Andy Zhou, Yi Zeng, Zinan Lin, Dawn Song, and Bo Li. 2024. RedCode: Risky Code Execution and Generation Benchmark for Code Agents. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-3369
 - [23] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 64 (2024), 23 pages. doi:10.1145/3660771
 - [24] Adnan Khan. 2026. Clinejection: Compromising Cline’s Production Releases Just by Prompting an Issue Triager. <https://adnanthekhan.com/posts/clinejection/>. Original research disclosure.
 - [25] Juhee Kim, Xiaoyuan Liu, Zhun Wang, Shi Qiu, Bo Li, Wenbo Guo, and Dawn Song. 2026. The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey. arXiv:2603.11088 <https://arxiv.org/abs/2603.11088v1> Preprint, version 1.
 - [26] Daoyuan Li. 2026. RepoGuardBench: Repository-Borne Prompt Injection Attacks and Lightweight Defenses for Local Coding Agents. 5th Deep Learning for Code Workshop at ICML. <https://github.com/DaoyuanLi2816/RepoGuardBench/blob/main/paper/main.pdf> Non-archival workshop presentation; official evaluation artifacts.
 - [27] Junjie Li, Xi Xiao, Yunbei Zhang, Chen Liu, Lin Zhao, Xiaoying Liao, Yingrui Ji, Janet Wang, Yingqiang Ge, Weijie Xu, Xi Fang, Xiang Xu, Tianchen Zhao, Youngeun Kim, Jihun Hamm, Tianyang Wang, and Chandan K. Reddy. 2026. Agent Harness Engineering: A Survey. <https://picrew.github.io/LLM-Harness/main.pdf> Survey preprint.
 - [28] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. 2026. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses. arXiv:2604.25850 <https://arxiv.org/abs/2604.25850>
 - [29] Chengzhi Liu, Yichen Guo, Yepeng Liu, Yuzhe Yang, Qianqi Yan, Xuandong Zhao, Wenyue Hua, Sheng Liu, Sharon Li, Yuheng Bu, and Xin Eric Wang. 2026. Auditing Agent Harness Safety. arXiv:2605.14271 <https://arxiv.org/abs/2605.14271>
 - [30] Fang Liu, Tianze Wang, Li Zhang, Zheyu Yang, Jing Jiang, and Zian Sun. 2025. Explainable Fault Localization for Programming Assignments via LLM-Guided Annotation. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2008–2020. doi:10.1109/ASE63991.2025.00167
 - [31] Yue Liu, Yanjie Zhao, Yunbo Lyu, Ting Zhang, Haoyu Wang, and David Lo. 2025. “Your AI, My Shell”: Demystifying Prompt Injection Attacks on Agentic AI Coding Editors. arXiv:2509.22040 <https://arxiv.org/abs/2509.22040v1>
 - [32] Ari Marzouk. 2025. IDEsaster: A Novel Vulnerability Class in AI IDEs. <https://maccarita.com/posts/idesaster/>. Original research disclosure, December 6, 2025.
 - [33] Microsoft. 2026. Understand Agent Harnesses. <https://code.visualstudio.com/docs/agents/concepts/agent-harnesses>. Visual Studio Code documentation; updated September 30, 2026; accessed October 2, 2026.
 - [34] Microsoft. 2026. Visual Studio Code Source Repository. <https://github.com/microsoft/vscode/commit/735c7376c1febaa709374e830fa9a2f0566c7092>. Pinned RQ1 revision 735c7376c1febaa709374e830fa9a2f0566c7092.
 - [35] Model Context Protocol. 2025. Model Context Protocol Specification. <https://modelcontextprotocol.io/specification/2025-11-25>. Specification version dated 2025-11-25.
 - [36] Yuting Ning, Zhehao Zhang, Yash Kumar Lal, Boyu Gou, Junyi Li, Weitong Ruan, Chentao Ye, Rahul Gupta, Diyi Yang, Yu Su, and Huan Sun. 2026. SkillHarm: Lifecycle-Aware Skill-Based Attacks via Automated Construction. arXiv:2606.02540 <https://arxiv.org/abs/2606.02540v1> Version 1.
 - [37] OpenAI. [n. d.]. Codex: AI Coding Agent. <https://openai.com/codex/>. Accessed October 2, 2026.
 - [38] OpenAI. 2026. OpenAI and Hugging Face Partner to Address Security Incident During Model Evaluation. <https://openai.com/index/hugging-face-model-evaluation-security-incident/>. Published July 21, 2026; subsequent updates through August 26, 2026.
 - [39] OWASP Gen AI Security Project. 2025. OWASP Top 10 for Agentic Applications 2026. <https://genai.owasp.org/download/52117/>. Version 2026, December 2025.
 - [40] peepepopapeepepo. 2025. Security Bug: Denied Commands in toolsSettings Are Not Properly Enforced. <https://github.com/aws/amazon-q-developer-cli/issues/2477>.
 - [41] Wanying Qu, Qinghua Mao, Yu Li, Jiyao Liu, Xin Zhang, Dadi Guo, Yanxu Zhu, Qingyu Liu, Leitao Yuan, Xi Lin, Shanfeng Zhu, Yanwei Fu, Jing Shao, Xia Hu, and Dongrui Liu. 2026. SHE: Trajectory-driven Safety Harness Evolution for LLM Agents. arXiv:2608.09885 <https://arxiv.org/abs/2608.09885>
 - [42] Yubin Qu, Yi Liu, Tongcheng Geng, Gelei Deng, Yuekang Li, Leo Yu Zhang, Ying Zhang, and Lei Ma. 2026. Supply-Chain Poisoning Attacks Against LLM Coding Agent Skill Ecosystems. arXiv:2604.03081 [cs.CR] <https://arxiv.org/abs/2604.03081v1>
 - [43] Qwen Code Contributors. 2026. Qwen Code Settings. <https://github.com/QwenLM/qwen-code/blob/a8e0d293fb0a80d790175789ea0ffefec506508/docs/users/configuration/settings.md>

- [44] Mohammadreza Rashidi. 2026. The Balkanization of Execution-Security Research for AI Coding Agents: Isolation, Access Control, and Time-of-Check-to-Time-of-Use Vulnerabilities. arXiv:2607.05743 <https://arxiv.org/abs/2607.05743v1> Preprint, version 1.
- [45] Jiajun Ruan, Peiyang Li, Yukun Chen, Fengting Li, and Chao Feng. 2026. Beyond Handcrafted Security: Towards Self-Evolving Defense for LLM Agents. arXiv:2608.12977 <https://arxiv.org/abs/2608.12977>
- [46] Jerome H Saltzer and Michael D Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939
- [47] David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi, and Maksym Andriushchenko. 2026. Skill-Inject: Measuring Agent Vulnerability to Skill File Attacks. arXiv:2602.20156 <https://arxiv.org/abs/2602.20156v1>
- [48] Ankur Singh, Jinqiu Yang, and Tse-Hsun Chen. 2026. IssueTrojanBench: Benchmarking AI Coding Agents Against Malicious Issue Requests. arXiv:2607.20759 <https://arxiv.org/abs/2607.20759v1>
- [49] Debu Sinha. 2026. CodeIPI: Indirect Prompt Injection for Coding Agents. Inspect Evals software benchmark. https://ukgovernmentbeis.github.io/inspect_evals/evals/ipi_coding_agent/
- [50] Anselm L. Strauss and Juliet M. Corbin. 1998. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory* (2 ed.). SAGE Publications, Thousand Oaks, CA, USA.
- [51] The Terminal-Bench Team. 2026. Terminal-Bench Task Repository. <https://github.com/harbor-framework/terminal-bench/tree/v4.0.0/tasks> Official task sources: mvcc-lsm-compaction and embedding-drift-monitor.
- [52] Naman Vats and Oleg Golev. 2026. The Scaffold Effect in Coding Agents: Harness Choice as a Hidden Variable in Coding-Agent Evaluation. arXiv:2607.22585 [cs.AI] <https://arxiv.org/abs/2607.22585> Preliminary work; under review at the DL4C workshop at ICML 2026.
- [53] Bowen Wei, Yunbei Zhang, Jinhao Pan, Kai Mei, Xiao Wang, Jihun Hamm, Ziwei Zhu, and Yingqiang Ge. 2026. ClawSafety: “Safe” LLMs, Unsafe Agents. arXiv:2604.01438 <https://arxiv.org/abs/2604.01438>
- [54] Yuchong Xie, Zesen Liu, Mingyu Luo, Zhixiang Zhang, Kaikai Zhang, Zongjie Li, Ping Chen, Shuai Wang, and Dongdong She. 2025. QueryIPI: Query-agnostic Indirect Prompt Injection on Coding Agents. arXiv:2510.23675 <https://arxiv.org/abs/2510.23675v1>
- [55] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-1601
- [56] Yu Yang, Zhenyuan Li, Xiandong Ran, Jiahao Liu, Jiahui Wang, Bo Yu, and Shouling Ji. 2026. Towards Scalable and Interpretable Mobile App Risk Analysis via Large Language Models. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*. <https://arxiv.org/abs/2508.15606>
- [57] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. In *The Thirteenth International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/5750f91d8fb9d5c02bd8ad2c3b44456b-Abstract-Conference.html
- [58] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2025. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 3827–3844. <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag>